

File System 1

Leonardo Ciscato Pajello

June 30, 2025

Contents

1	Il File System (FS)	2
1.1	File	2
1.1.1	Struttura dei file	2
1.1.2	Attributi dei file	3
1.2	Operazioni sui file	3
1.2.1	File aperti	4
1.2.2	Lock dei file aperti	4
1.2.3	Tipi di file	5
1.2.4	Supporto alla struttura dei file	5
1.2.5	Struttura interna dei file	6
1.2.6	Modalità di accesso	6
1.3	Directory	6
1.3.1	Info e operazioni sulle Directory	7
1.3.2	Directory a un livello	7
1.3.3	Directory a due livelli	7
1.3.4	Directory con struttura ad Albero	8
1.3.5	Directory a grafo aciclico	8
1.3.6	Directory a grafo generico	9
1.4	Montaggio di un FS	10
1.4.1	Procedura di montaggio	10
1.4.2	Considerazioni	10
1.5	File Sharing	11
1.5.1	File Sharing multiutente	11
1.6	File System remoti	11
1.7	Malfunzionamenti	12
1.8	Semantica della coerenza	12
1.8.1	Protezione nel File System	13
1.9	Caso UNIX	13
1.9.1	Review del caso di UNIX	13
1.9.2	UNIX	13
1.9.3	Unix: Operazioni su file e directory	14
1.9.4	Unix: links	14
1.9.5	Protezione in UNIX	15

1 Il File System (FS)

Il **Sottosistema per la gestione dei file** è la parte più visibile di un OS. Questo fornisce meccanismi per registrazione, accesso e protezione di dati/programmi; visualizza i dispositivi di memorizzazione come HDD, SSD, USB ecc... E astrae la complessità di utilizzo dei diversi mezzi di memorizzazione secondaria fornendo una visione logica e metodi di accesso uniformi. La struttura logica con cui l'OS gestisce i file prende il nome di FS, l'organizzazione gerarchica con cui l'OS gestisce l'archiviazione dei file (HFS - Hierarchical File System) non ha alcun legame con la posizione fisica dei file in mass memory.

Il **File system** è responsabile della gestione dei file in mass memory:

- struttura i dati in files;
- li organizza in dir;
- fornisce a User un insieme di funzioni di alto livello per operare su questi, mascherando le reali operazioni di basso livello (Malloc);

Il FS, garantisce una gestione dei file indipendente dalle caratteristiche fisiche dei dispositivi che costituiscono la MassMem.

1.1 File

Si definisce **FILE** un insieme di informazioni correlate e registrate nella memoria secondaria, a cui è stato assegnato un nome.

Dal punto di vista USER:

- è la più piccola parte di memoria secondaria indirizzabile logicamente;
- i dati possono essere scritti nella memoria secondaria soltanto all'interno di un file;

Dal punto di vista OS: i file vengono mappati su dispositivi fisici di memorizzazione non volatili.

Tipi di files:

- **Dati:** numerici, alfanumerici, binari;
- **Programmi:** Codice sorgente, codice oggetto, codice eseguibile;

1.1.1 Struttura dei file

Se non è presente il file è una semplice sequenza di byte.

Sequenza di Record:

- Linee;
- Record a lunghezza fissa;
- Record a lunghezza variabile

Struttura complessa:

- Documento formattato;
- File eseguibile rilocabile;

La struttura viene decisa o dall'OS o dall'applicativo che crea il file, in generale un file è formato da una sequenza di bit, byte, righe o record, il cui significato è definito dal creatore del file stesso.

Il **tipo** del file è ciò che solitamente ne definisce la struttura, tutti gli OS devono prevedere un tipo di file: **.exe**. Alcuni Sistemi prevedono la gestione diretta di diverse strutture file e mettono a disposizione operazioni specifiche

1.1.2 Attributi dei file

I seguenti:

- **Nome:** identificativo del file per lo user;
- **Identificativo:** etichetta unica (numero) che identifica il file all'interno del FS;
- **Tipo:** necessario per i sistemi che supportano tipi differenti;
- **Locazione:** puntatore al dispositivo e alla posizione del file sul dispositivo;
- **Protezione:** parametri di controllo per l'accesso in lettura, scrittura ed exec;
- **Ora, data e ID utente:** dati necessari alla sicurezza del sistema e per il controllo d'uso;

Le info sui file sono conservate nella struttura di **Directory** che risiede sulla memoria secondaria.

1.2 Operazioni sui file

Un file è un **tipo di dato astratto** su cui sono definite le operazioni di:

- Creazione;
- Lettura / Scrittura;
- Posizionamento nel file (**file seek**);
- Cancellazione;
- Troncamento;
- Impostazione degli attributi;

Creazione: riempimento del file nel FS, rilascia lo spazio allocato al file e elimina il corrispondente elemento della dir.

Troncamento: cancellazione del contenuto del file, ma parte degli attributi restano immutati.

impostazione degli attributi: riempimento ed aggiornamento del relativo elemento di dir.

Le operazioni su file richiedono solitamente una ricerca dell'elemento associato al file all'interno della struttura delle dir. L'OS mantiene in mem centrale una tabella contenente info su tutti i file aperti: **tabella dei file aperti**

Una volta richiesta un'operazione su file si ricercano le info relative tramite un indice nella tabella sopracitata, quando il file non è più in uso deve essere chiuso e l'OS rimuove l'elemento relativo nella tabella.

Apertura e chiusura file:

- $\text{open}(F_i)$: ricerca nella dir su disco l'elemento F e ne copia il contenuto nella tabella file aperti restituendo un puntatore all'elemento tabulato;
- $\text{close}(F_i)$: copia il contenuto dell'elemento F residente in memoria principale nella sua dir su disco e lo rimuove;

1.2.1 File aperti

Nei sistemi multiuser di cono due lvl di tabelle:

Tabella di Sistema e Tabella associata a processo

Sistema:

- riferimenti a tutti i file aperti nel sistema;
- posizione e dimensione del file nel disco;
- date di ultimo accesso/ultima modifica;
- counter di aperture;

Processo:

- riferimenti a tutti i file aperti dal processo;
- puntatore alla posizione corrente nel file;
- diritti di accesso e info di contabilizzazione;
- gli elementi della tabella dei file associata al processo puntano al relativo elemento contenuto nella tabella di sistema;

Da notare bene che:

Puntatore alla posizione corrente nel file: manda all'ultima locazione dove è stata realizzata un'operazione di R/W per ogni processo che ha aperto il file.

Contatore di aperture: torna il numero di processi che hanno aperto il file, per rimuovere i dati dalla tabella qualora venga chiuso il file.

Locazione del file su disco: cache delle info di accesso ai dati permanenti.

Diritti di accesso: controllati dall'OS per permettere o negare le operazioni di I/O richieste.

1.2.2 Lock dei file aperti

Questa feature é garantita da qualche OS e FS, offre una mediazione per l'accesso condiviso a file. Obbligatorio o Consigliato:

- **Obbligatorio:** l'accesso a file viene negato se il lock è già stato acquisito da un altro processo (Windows);
- **Consigliato:** i processi trovano ch elo stato di un file é bloccato e decidono su cosa fare (UNIX);

Se il lock è obbligatorio l'OS assicura integrità dei dati soggetti a lock. Se invece è consigliato il programmatore deve garantire la corretta acquisizione e cessione di lock.

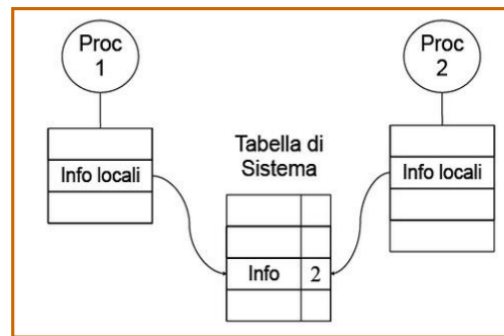


Figure 1: Una entry dalla tab di sistema può essere rimossa quando il counter = 0

1.2.3 Tipi di file

Il tipo di file è un attributo che ne indica la struttura logica e permette di interpretarne correttamente il contenuto, alcuni OS gestiscono diversi tipi di file. Conoscendo il tipo si possono evitare errori comuni quali stampare un file.exe.

Un OS che riconosce il tipo del file lo può manipolare in modo "mirato". Tre sono le tecniche di gestione dei tipi di file:

- **Meccanismo delle estensioni:** il tipo è indicato da suffisso del nome;
- **Attributo "tipo" associato al file nella dir=** MAC OS;
- **Magic number:** il tipo è rappresentato da un valore posto all'inizio del file (UNIX).

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information

Esempi:

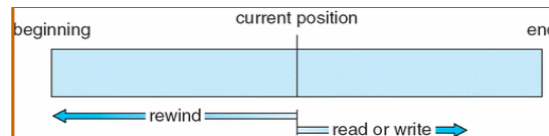
- In **MS-DOS** possono essere eseguiti solo: .COM, .EXE, .BAT;
- Su **Apple** ogni file possiede un attributo di creazione indicativo al programma che lo ha creato;
- **UNIX** memorizza un *magic number* per indicare il tipo di file, usa le estensioni solo come suggerimento.

1.2.4 Supporto alla struttura dei file

Il tipo di un file e la corrispondente struttura logica sono riconosciuti e gestiti in modi diversi dai vari OS. Se questi gestiscono molti formati il codice di sistema è più ingombrante, ci potrebbe essere incompatibilità tra programmi con file di formato differente ma hanno una gestione efficiente per i formati supportati. Altrimenti il codice sistema è più snello ma i formati devono essere gestiti dal programmatore.



Figure 2: Parte dell'ultimo blocco fisico contenente il file rimane inutilizzata: frammentazione interna



Esempio: Unix attua una scelta minimale, i file vengono considerati semplici stringhe di byte e solo i quelli eseguibili hanno un formato predefinito dall'OS.

1.2.5 Struttura interna dei file

La dimensione dei **blocchi** del disco, detti anche *record fisici* é fissa, occorre risolvere il problema della corrispondenza tra record logici e fisici (*packing*)

1.2.6 Modalità di accesso

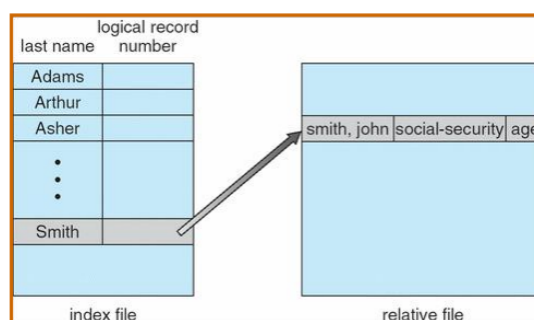
L'accesso **sequenziale** é un modello di accesso a file che si ispira al natstro. Si può accedere solo al record - blocco successivo:

L'accesso **diretto**: è un modello di accesso a file ispirato al disco, si può accedere ad ogni blocco - record tramite il numero di blocco relativo

L'accesso **indicizzato** può essere realizzato sulla base del metodo ad accesso diretto, implica la costruzione di un file indice per l'accesso. Questo indice contiene puntatori ai blocchi, per reperirne un elemento occorre cercare il puntatore realizzato interno all'indice.

1.3 Directory

Si può interpretare directory come una tabella che ha un elemento per ogni nome di file, questo elemento permette di reperire gli attributi del file e le info per accedervi al contenuto. Come i file anche le directory risiedono su disco



1.3.1 Info e operazioni sulle Directory

Informazioni:

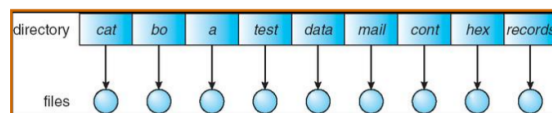
- Nome, Tipo, Indirizzo;
- Lunghezza attuale e massima;
- Data ultimo accesso e ultima modifica;
- ID proprietario e info di protezione;

Operazioni:

- Ricerca di un file;
- Creazione / Cancellazione di un file;
- Elenco dei contenuti di una dir;
- Ridenominazione di file;
- Attraversamento del file system;

1.3.2 Directory a un livello

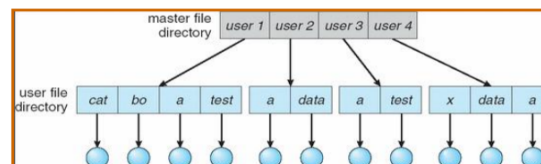
Una directory unica per tutti gli utenti



- Tutti i file file di tutti gli utenti in un'unica directory
- Problemi di naming (va scelto un nome diverso per ogni file)
- Nessun raggruppamento logico
- Problemi accentuati con numero grande di file

1.3.3 Directory a due livelli

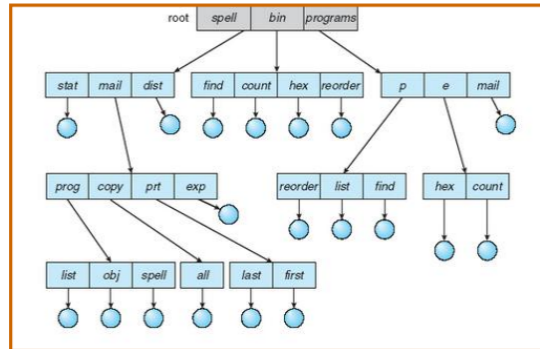
Directory separate per ciascun utente



- Ammessi nomi uguali per file di utenti diversi
- Ricerca più efficiente
- Nessuna capacità di raggruppamento logico (solo in base a proprietari) Permette a un utente di individuare file di altri utenti (pathname: utente/file)
- Ricerca dei FS: percorso di ricerca
- Quando si apre sessione di lavoro di ricerca nella Master File Directory l'ID utente, ammesso poi al sistema all'interno della propria User File Directory
- Ogni riferimento a file da parte dell'utente viene interpretato da SO come esclusivamente correlato a file presenti nella relativa UFD

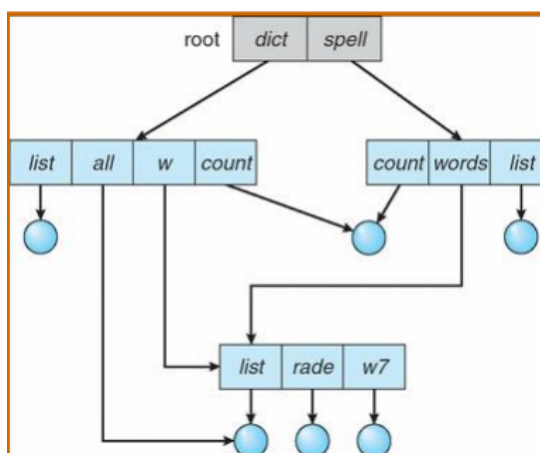
- Per riferirsi a file di altri utenti (se l'accesso è autorizzato) ogni utente deve utilizzare pathname completo dei file (nome utente, nome file)
- FS vengono raccolti in opportune directory raggiungibili da tutti gli utenti

1.3.4 Directory con struttura ad Albero



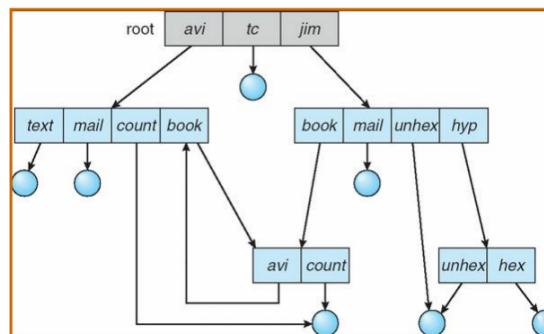
- Ricerca efficiente
- Capacità di raggruppamento logico
- Concetto di pwd (directory corrente)
- Pathname **assoluto** o **relativo**
- Creazione di nuovo file o cancellazione di un file esistente viene effettuata nella pwd
- Creazione di nuova directory viene interpretata come creazione di sottodirectory della pwd
- Cancellazione di una directory ammessa solo se directory è vuota oppure cancella anche tutto il suo contenuto

1.3.5 Directory a grafo aciclico



- Presenza di file e sottodirectory condivisi
- **Aliasing**: più nomi diversi possono essere utilizzati per identificare stesso oggetto
- Condivisione può essere implementata mediante **link** o per duplicazione dell'elemento di directory
- **Problema**: se si cancella *w/list* → puntatore dangling
- **Soluzioni**:
 - Usare puntatori all'indietro che consentano reperimento e cancellazione di tutti i link a file cancellato
 - Conservazione del file fino a che non esistano più link (lista dei riferimenti a file, numero di riferimenti: quando contatore è 0 il file può essere cancellato)
- Vi sono **”nuovi oggetti”** contenuti nelle directory e necessitano di opportune operazioni:
 - Creare/cancellare un **link**, cioè un nome diverso (per un file già esistente)
 - **Risolvere un link** = seguire il puntatore per reperire il file

1.3.6 Directory a grafo generico



Evitare cicli o ammettere cicli?

- Mantenere grafo aciclico garantisce semplicità degli algoritmi necessari per attraversarlo
- Proibire cicli significa ammettere link a file ma non a directory
- Permessi link a directory, ma ad ogni aggiunta di link si deve verificare assenza di cicli
- Altrimenti di devono gestire i cicli → **Garbage collection**: per implementare cancellazione di parti non più riferite si attraversa FS marcando i file accessibili e in un secondo passaggio si rimuovono i file non marcati

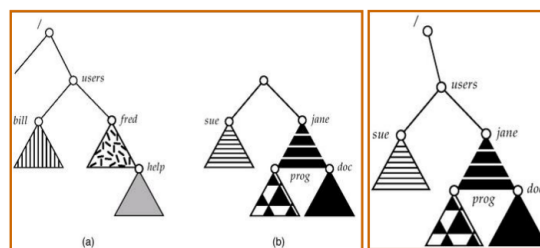
1.4 Montaggio di un FS

- Un FS deve essere **montato** prima di poter essere acceduto da processi di un sistema
- Un FS *unmounted* può essere montato ad un **mount point** prescelto
- Alcuni S.O. richiedono FS di tipo prefissato, altri ne supportano diversi e sondano strutture del dispositivo/i per determinare tipo del/i file system presente/i (o richiedono all'utente di specificare il tipo)

1.4.1 Procedura di montaggio

1. Si fornisce a S.O. nome del dispositivo da montare e locazione che dovrà assumere nella struttura del FS (solitamente punto di montaggio è una directory vuota cui sarà agganciato il FS da montare)
2. Una volta montato il FS risulta accessibile a programmi utente in modo trasparente e diventa parte integrante del grafo delle directory
3. Directory su cui viene montato un FS può non essere vuota: nel momento in cui si effettua il montaggio i dati ivi contenuti non risultano visibili fino all'unmount

Esempio: Si effettua montaggio su `/users` del FS in figura (b) identificato (per esempio) da `/device/usr` → le home degli utenti *bill* e *fred* diventano inaccessibili



1.4.2 Considerazioni

- Sistemi Mac e Windows rilevano tutti i dispositivi all'avvio della macchina e montano automaticamente tutti i FS in essi contenuti
- Nei sistemi UNIX-like i FS devono essere montati esplicitamente:
 - Un file di conf. del sistema contiene una lista di dispositivi e relativi punti di montaggio da montare automaticamente all'avvio; mount può essere effettuato anche durante sessione di lavoro:
\$ mount /dev/usr /users
\$ unmount /dev/usr
 - in Linux: `/etc/fstab` è il file di configurazione di sistema per descrizione statica dei dispositivi di memoria collegati al pc il cui mount deve essere effettuato all'avvio

1.5 File Sharing

- Il file sharing permette la collaborazione tra utenti/processi e riduce le risorse richieste per raggiungere un obiettivo di calcolo
- Sono necessari meccanismi di protezione per garantire il controllo dell'accesso ai file mediato dall'OS
- Nei sistemi distribuiti la condivisione avviene attraverso la rete
- **NFS** (Network File System) permette di realizzare la condivisione di file in ambienti distribuiti

1.5.1 File Sharing multiutente

Modello che utilizza i concetti di owner e di group di user "affini" all'owner:

- l'owner è lo user che possiede il file e che può cambiare gli attributi del file (o dir)
- un group definisce il sottoinsieme di user autorizzati a condividere l'accesso a file
- gli identificatori del group (**GID**) e del proprietario (**UID**) di un file (o dir) sono memorizzati come attributi del file (nel relativo file dir)

1.6 File System remoti

Uso della rete per ottenere accesso a file residenti su sistemi remoti, si può realizzare in diversi modi:

- **Trasferimento**
- **DFS** (File system distribuito), che permette la visibilità e l'accesso (automatico) dal calcolatore locale ad una remote dir
- accesso tramite **browser** (semi-automatico) **FTP**, anonimo, non è esplicito ma è *wrapped*, nell'operazione di download

Nel **modello client-server**, il server mette a disposizione risorse (sotto forma di dir o file) ai client che ne fanno richiesta.

- **Modello multi-a-molti**: un server può gestire richieste provenienti da più client, il client può accedere a più server
- Problemi di **autenticazione**:
 - Protocolli insicuri che possono condurre a spoofing
 - Sicurezza ottenuta mediante autenticazione reciproca di client e server tramite key di cifratura

Possibili problemi: compatibilità fra client e server (relativamente all'algoritmo di cifratura) e scambio sicuro delle chiavi.

NFS è il protocollo client-server standard di UNIX:

- gli **UID** devono coincidere nel client e nel server
- dopo il montaggio del file system remoto, le richieste di accesso a file vengono inviate al server attraverso la rete
- il server applica i normali **controlli di accesso** e, qualora producano un risultato positivo, restituisce un ***file handle*** al client, che lo usa per eseguire le successive operazioni su file

CIFS (Common Internet File System) equivalente Windows del protocollo NFS

Le system call locali vengono tradotte in chiamate (per gli stessi servizi) su file system remoti.

1.7 Malfunzionamenti

- Nei file system locali possono verificarsi malfunzionamenti dovuti a:
 - problemi hardware dei disks/devices
 - alterazioni dei **metadati** (informazioni per il reperimento dei file nelle dir)
 - problemi ai controller dei dischi
 - problemi alle connessioni hardware
 - errori umani
 - ...
- Nei remote file system i malfunzionamenti possono avvenire anche per cadute di rete o dei remote server sui quali sono presenti i file
- Per il ripristino da malfunzionamenti è necessario mantenere delle **informazioni di stato** sia sui client che sui server (per poter far riprendere entrambi)
- I protocolli attualmente più diffusi non mantengono informazioni di stato (es. NFS che trasferisce tutte le informazioni nella singola richiesta con alta tolleranza ai guasti ma scarsa sicurezza)

1.8 Semantica della coerenza

Semantica che specifica quando le modifiche apportate da un utente ai dati contenuti in un file possano essere osservate da altri utenti, è direttamente correlata agli algoritmi di sincronizzazione fra processi, ma risulta meno critica per via delle lunghe latenze e basse velocità di trasferimento dei dischi e della rete.

La semantica UNIX impone:

- le scritture in un file aperto da un utente siano immediatamente visibili agli altri utenti che hanno aperto lo stesso file
- la condivisione del puntatore alla posizione corrente del file (il file ha una singola immagine e tutti gli accessi si alternano, intercalandosi, a prescindere dalla loro origine
- la contesa per l'immagine fisica del file determina il differimento dei processi utente

1.8.1 Protezione nel File System

- Per salvaguardare le informazioni si devono gestire le eventualità di:
 - danni fisici (**affidabilità**) assicurata dalla *ridondanza*
 - accessi impropri (**protezione**) ottenuta mediante *controllo degli accessi*
- Il proprietario del file deve poter controllare:
 - quali tipi di accesso sono leciti sul file
 - chi può accedere al file
- I vari tipi di accesso a file sono:
 - Read
 - Write
 - Esecuzione
 - Append
 - Cancellazione
 - Elencazione degli attributi
- Il controllo degli accessi è attuabile usando:
 - Liste di controllo accessi [al file si associano gli user e i tipi di accesso autorizzati per ogni user]
 - Liste di controllo accessi per classi di utenti [es. owner, group, others]
 - Associazione di una password a ciascun file

1.9 Caso UNIX

1.9.1 Review del caso di UNIX

File e dir possono appartenere a più dir, costruendo una struttura a *grafo generale*.

Varie dir ospitano programmi di sistema, per esempio:

- **/dev** per i device driver
- **/bin** per il codice eseguibile
- **/usr/include** per le librerie di sistemi
- **/etc** per i file di configurazione

A ciascun utente è associata una dir chiamata *home directory*, solitamente subdir di **/usr** o di **/home** e lì andranno i file creati dallo user.

1.9.2 UNIX

- Ciascun file è identificato da un **pathname assoluto** che include l'intero cammino dalla root al file stesso
- Ciascun pathname è unico (file e subdir → nomi distinti)
- Uno user che interagisce con il file system ha una propria **pwd** (present working directory) [non serve specificare il path fino alla pwd - **relative pathname**]
- Il simbolo **."** fa riferimento alla pwd, mentre **.."** fa riferimento alla dir contenente la pwd

1.9.3 Unix: Operazioni su file e directory

Comandi per la manipolazione di file e dir:

- **chmod** - stabilisce i parametri di protezione del file
- **chown** - cambia il proprietario di un file
- **cmp** - confronta byte a byte due file
- **cp** - copia un file in un altro (situato in un'altra dir o nella stessa ma con nome diverso)
- **find** - trova file per nome o per altre caratteristiche
- **grep** - ricerca un file attraverso una stringa contenuta in esso
- **ln** - crea un link ["ln oldfile newlink" crea un nuovo elemento della pwd, newlink, relativo ai contenuti del file oldfile]
- **more** - visualizza il contenuto di un file (su terminale per default)
- **mv** - muove un file (serve anche per rinominarlo)
- **rm** - cancella un file
- **mkdir** - crea una dir
- **cd** - cambia dir (contesto)
- **ls** - elenca i file e le subdir contenute nella dir specificata
- **pwd** - mostra il pathname della pwd
- **rmdir** - rimuove una dir
- **mount** - monta un file system
- **man** - permette di consultare il manuale

1.9.4 Unix: links

- Un **hardlink** è un'etichetta o un nome assegnato ad un file
- Si possono avere nomi distinti che identificano lo stesso file
- I comandi eseguiti relativamente ad uno qualsiasi dei nomi di un file operano sugli stessi "contenuti"
- Qualsiasi cambiamento effettuato su oldfile riguarda newlink (e viceversa)
- il comando rm può essere usato anche per rimuovere un hardlink (dopo la rimozione del link il contenuto del file viene mantenuto fino a che c'è almeno un nome che si riferisce a quel file)
- Un link simbolico o **softlink** è un tipo speciale di file che punta ad un altro file
- Diversamente agli hardlink non contiene i dati relativi al target file, semplicemente punta ad un diverso elemento di una dir

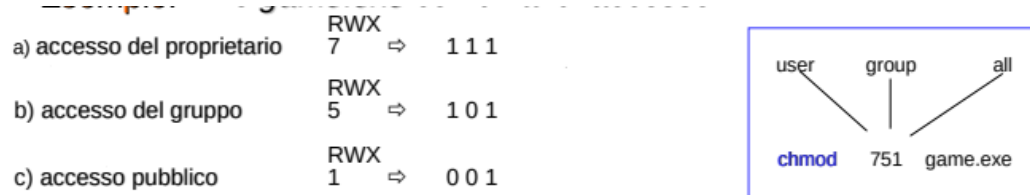
- In seguito alla cancellazione, i softlink diventano inutilizzabili(**dangling**, mentre gli hardlink preservano (finchè ne esiste almeno uno) il contenuto del file
- Per creare un link simbolico si usa: "ln -s filename mylink"

1.9.5 Protezione in UNIX

Alla creazione di un file, l'utente specifica la **protezione** da applicarci, indica quindi quali operazioni possono essere svolte e da chi. I UNIX:

- Tipo di accesso: read, write, exec, append, delete, attribute list;
- Metodo di accesso: read, write, exec;
- Tre classi User: user, group all;

esempio:



Il file può essere R/W/E dal proprietario, R/E da utenti del gruppo, E da tutti gli altri user di sistema.

Come per i file, a ciascuna dir sono associati tre campi (user, group, all) ciascuno composto da tre bit rwx. Un utente può elencare il contenuto di una dir solo se il bit r è inserito nel campo appropriato, modificarla se w è impostato e accedervi se x è settato.